

# **ClimateQuest –**

## **Entwicklung und Veröffentlichung einer App zur Förderung nachhaltigen Handelns**

Stand: 20.04.2026



## Projektüberblick

Das Ziel meines Projektes ist es, eine App zu programmieren, welche User dazu motiviert, privat mehr für den Klimaschutz zu tun. Um dies zu gewährleisten, soll die App folgende Funktionen haben:

- **Registrierung, E-Mail-Adressen Verifizierung, Login, Passwort-Reset:** Dies sind die grundlegenden Funktionen der meisten Apps.
- **Erstellen von Aktionen und Erhalten von Klimapunkten und Leveln:** Indem User Aktionen eintragen, die sie durchgeführt haben und die dem Klima helfen, erhalten sie Klimapunkte. Je mehr Klimapunkte ein User bereits gesammelt hat, desto höher ist auch sein Level. Dadurch steigt die Motivation, mehr für das Klima zu tun.
- **Erstellen, Beitreten, Einsehen von Families:** Indem sich mehrere User, beispielsweise Kollegen, Freunde, Klassenkameraden oder Verwandte, zu einer Family zusammenschließen, können sie sich in Rankings vergleichen. Auch hierdurch werden User motiviert, mehr Klimaschutz zu betreiben, um höher im Ranking zu stehen.
- **Erstellen, Beitreten, Einsehen von Communities:** Mehrere Families können sich zu einer Community zusammenschließen. Wie bei den Families gibt es hier ein Ranking, welches den Durchschnitt der Families auflistet. Hierbei steigt ebenfalls die Motivation, mehr für den Klimaschutz zu tun, indem man nicht nur für sich selbst Aktionen durchführt, sondern für die gesamte Family.
- **Erstellen von Events:** Ein Event ist eine Seite auf *ClimateQuest*, die jeder User erstellen und auf die jede Person zugreifen kann. Hierbei wird zu einem gemeinschaftlichen Event aufgerufen, beispielsweise Müll sammeln am dd.mm.yyyy, Beginn um xx Uhr in der Straße xy. Andere User können hierzu Fragen stellen und dem Event beitreten, also mitteilen, dass sie teilnehmen werden. Indem man Events erstellt, kann man die Reichweite und Organisation größerer Events vereinfachen: Man weiß nicht nur, wie viele Personen ungefähr kommen werden, alle Informationen können auch noch von allen Menschen auf einen Blick gefunden werden und Updates werden direkt kommuniziert.
- **Erstellen von Artikeln:** In Artikeln kann man sein eigenes Wissen zu Klimaschutzthemen mit anderen teilen.
- **Erstellen von Forums-Beiträgen:** Wenn man Fragen beispielsweise zur Organisation von Events o.Ä. hat, kann man diese Fragen im Forum stellen, sodass andere User, die vielleicht schon mehr Erfahrung mit diesem Thema gemacht haben, den Fragenden helfen können. So können sich User zu verschiedenen Klimaschutzthemen vernetzen.
- **Erstellen von Petitionen:** Es gibt auch die Möglichkeit, Petitionen zu erstellen, um wirtschaftliche oder politische Veränderungen zu erreichen.
- **Dashboard:** Hier kann eine Übersicht über eingetragene Aktionen, gesammelte Klimapunkte, das eigene Level sowie die Families und Communities, in denen man Mitglied ist, gefunden werden.
- **Wöchentliches Ziel:** Um die Motivation für mehr Klimaschutz noch weiter zu erhöhen, gibt es ein wöchentliches Ziel, welches jeder User selbst festlegen kann. Im Dashboard sieht man, wieviele Klimapunkte einem noch fehlen, bis dieses Ziel erreicht wird.
- **Streak:** Wie bei vielen anderen Apps gibt es auch hier eine Streak, die anzeigt, wie oft hintereinander das wöchentliche Ziel erreicht wurde.

- **Baumpflanzungen:** Sobald 1000 Klimapunkte erreicht wurden, erhält man einen Credit. Dieser kann ausgegeben werden, sodass von Plant-for-the-Planet, mit denen ich eine Kooperation eingegangen bin, ein Baum in Kenia gepflanzt wird.
- **Tracking:** Aktionen, die jede Woche durchgeführt werden, *1 Woche vegetarisch ernähren*, können automatisch eingetragen werden. Zudem kann man mittels GPS tracken, wie viele Kilometer man beispielsweise mit dem Fahrrad statt mit dem Auto gefahren ist.

Die App ist unter <https://climate-quest.de> verfügbar, der Code kann unter <https://github.com/Hipposmile/ClimateQuest> eingesehen werden.

# Inhaltsverzeichnis

|                                                         |           |
|---------------------------------------------------------|-----------|
| <b>PROJEKTÜBERBLICK .....</b>                           | <b>2</b>  |
| <b>INHALTSVERZEICHNIS.....</b>                          | <b>4</b>  |
| <b>FACHLICHE KURZFASSUNG .....</b>                      | <b>5</b>  |
| <b>1. MOTIVATION UND FORSCHUNGSZIEL .....</b>           | <b>5</b>  |
| <b>2. HINTERGRUND UND THEORETISCHE GRUNDLAGEN .....</b> | <b>5</b>  |
| <b>3. VORÜBERLEGUNGEN.....</b>                          | <b>7</b>  |
| 3.1. KOOPERATIONEN.....                                 | 7         |
| 3.2. AKTIONEN.....                                      | 8         |
| 3.3. DESIGN.....                                        | 9         |
| <b>4. PROGRAMMIERUNG.....</b>                           | <b>9</b>  |
| 4.1. ALLGEMEINES.....                                   | 9         |
| 4.2. FUNKTIONEN DER WEBSEITE .....                      | 10        |
| 4.3. PROBLEME BEIM PROGRAMMIEREN .....                  | 12        |
| <b>5. VERÖFFENTLICHUNG.....</b>                         | <b>12</b> |
| 5.1. SCHRITTE BEI DER VERÖFFENTLICHUNG.....             | 12        |
| 5.2. PROBLEME BEI DER VERÖFFENTLICHUNG .....            | 13        |
| 5.3. RECHTLICHES.....                                   | 13        |
| 5.4. SEO .....                                          | 14        |
| 5.5. PERFORMANCE .....                                  | 14        |
| 5.6. TESTEN.....                                        | 15        |
| 5.7. HACKERSICHERHEIT.....                              | 16        |
| 5.8. MOBILE APPS .....                                  | 16        |
| <b>6. ERGEBNISDISKUSSION, FAZIT, AUSBLICK.....</b>      | <b>18</b> |
| <b>7. QUELLEN UND UNTERSTÜTZUNGSLEISTUNGEN.....</b>     | <b>20</b> |
| 7.1. VERWENDETE BILDER UND GRAFIKEN.....                | 20        |
| 7.2. INTERNETSEITEN .....                               | 20        |

## Fachliche Kurzfassung

Mithilfe von Python, HTML, CSS und JavaScript habe ich eine Web-App namens *ClimateQuest* inklusive Datenbanksystem und Backend programmiert. Das Ziel dieser App ist es, Menschen zu mehr Klimaschutz zu motivieren. Hierzu habe ich ein Python-Framework zur Backend-Entwicklung namens Django verwendet, mit welchem auch bekannte Apps wie beispielsweise YouTube, Instagram, Spotify oder Dropbox geschrieben wurden. Veröffentlicht habe ich die App mit Gunicorn + Supervisor + Nginx auf einem Virtual Private Server (=VPS).

### 1. Motivation und Forschungsziel

Die Menschheit ist durch den anthropogenen Klimawandel in großer Gefahr. Aufgrund des enormen Ausstoßes von Treibhausgasen wie Kohlenstoffdioxid oder Methan erhitzt sich unsere Erde in beispielloser Geschwindigkeit. Dürren, aber auch Starkregenereignisse sind die Folge, was dazu führt, dass weite Landstriche entweder durch Wassermangel oder Überschwemmungen unbewohnbar werden. Extremwetterereignisse werden zur Normalität, Ernteauffälle deutlich häufiger, weshalb auch die Zahl der Klimaflüchtlinge steigt und weiter steigen wird. Daher ist ein wirkungsvoller Klimaschutz das zentrale Thema des 21. Jahrhunderts.

Obwohl die Industrie einen großen Teil der Treibhausgasemissionen verursacht, sind der Konsum und das Verhalten privater Haushalte für den Hauptteil der Emissionen verantwortlich: 2021 etwa waren Privatpersonen für etwa 71% der gesamtdeutschen Emissionen verantwortlich.<sup>1</sup> Das beweist: Obwohl oft das Gegenteil behauptet wird, haben Privatpersonen einen großen Einfluss auf den Klimawandel. Auch wenn viele Menschen das Gefühl haben, man könne selbst kaum etwas bewirken, ist das nicht der Fall: Indem man sich beispielsweise eine Solaranlage anschafft, senkt man nicht nur den eigenen CO<sub>2</sub>-Ausstoß, es steigt auch die Wahrscheinlichkeit, dass sich Nachbarn ebenfalls eine Anlage installieren. Zudem demonstriert man der Wirtschaft, dass nachhaltige Technologien gefragt sind.

Dennoch fehlt vielen Menschen die Motivation, tatsächlich etwas für den Klimaschutz zu tun. Aus diesem Grund habe ich mir eine Möglichkeit überlegt, dies zu ändern: Eine App, mit der Nutzerinnen und Nutzer eigene klimafreundliche Aktionen dokumentieren können und dafür *Klimapunkte* erhalten. Indem man ab einer bestimmten Anzahl von Klimapunkten ein höheres Level erreicht oder sich mit anderen Usern mithilfe von Rankings vergleichen kann, steigt die Motivation, dem Klima selbst aktiv zu helfen.

### 2. Hintergrund und theoretische Grundlagen

Es gibt bereits einige Apps, die User zu mehr Nachhaltigkeit im Alltag motivieren wollen. Meine App überschneidet sich mit diesen zwar in einigen Bereichen, soll aber auch neue Möglichkeiten schaffen.

---

<sup>1</sup> Umweltbundesamt, Treibhausgasemissionen stiegen 2021 um 4,5 Prozent, <https://www.umweltbundesamt.de/presse/pressemitteilungen/treibhausgasemissionen-stiegen-2021-um-45-prozent>, zuletzt aufgerufen: 22.11.2025; Statistisches Bundesamt, CO<sub>2</sub>-Fußabdruck der privaten Haushalte: 540 Millionen Tonnen, <https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Umwelt/UGR/energiefluesse-emissionen/emissionen-energiefluesse.html>, zuletzt aufgerufen: 22.11.2025

| APP                | FUNKTIONSWEISE                                                                                                                                                                                                                                                                                                                                                                                                                 | VORTEILE GEGENÜBER<br>CLIMATEQUEST                                                                                                                                                                                                                                 | NACHTEILE<br>GEGENÜBER<br>CLIMATEQUEST                                                                                                                                                                                                                                                                                                                                        |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>KLIMA-TALER</b> | <ul style="list-style-type: none"> <li>- Automatisches Tracken von klimafreundlichen Aktionen</li> <li>- Abfotografieren des Wasser- / Stromzählerstandes</li> <li>- Erhalten von Klimapunkten für 5 Kilogramm eingespartes CO<sub>2</sub></li> <li>- Einlösen von Klimatalern gegen kostenlose Eintritte / Vergünstigungen bei Schwimmbadbesuchen, lokalen Händler*innen, o.Ä. oder gegen CO<sub>2</sub> Ausgleich</li> </ul> | <ul style="list-style-type: none"> <li>- „Wert“ von Klimatalern (Vergünstigungen / kostenlose Eintritte oder CO<sub>2</sub> Ausgleich durch Ausgabe von Klimatalern)</li> <li>- Automatisches Tracken der Kilometer</li> <li>- Schummeln sehr erschwert</li> </ul> | <ul style="list-style-type: none"> <li>- Nur wenige Aktionen, welche man eintragen kann</li> <li>- Keine Möglichkeit, Aktionen manuell eintragen (kann auch als Vorteil angesehen werden → keine Möglichkeit, unehrlich zu sein)</li> <li>- Rankings (Families / Communities), Level, eigene Artikel, Events, Forum, Petitionen, wöchentliches Ziel, Streak fehlen</li> </ul> |
| <b>ECOHERO</b>     | <ul style="list-style-type: none"> <li>- Eintragen von klimafreundlichen Aktionen, ggf. inklusive eigener Beschreibung und Bild</li> <li>- Teilen dieser Aktionen über SocialMedia und Veröffentlichen bei EcoHero</li> <li>- Aufsteigen in andere Level</li> </ul>                                                                                                                                                            | <ul style="list-style-type: none"> <li>- Social Media stärker im Vordergrund (kann auch als Nachteil angesehen werden)</li> <li>- Teilen einzelner Aktionen</li> </ul>                                                                                             | <ul style="list-style-type: none"> <li>- Rankings (Families / Communities), Level, Artikel, Events, Forum, Petitionen, wöchentliches Ziel, Streak fehlen</li> </ul>                                                                                                                                                                                                           |
| <b>EARNEST</b>     | <ul style="list-style-type: none"> <li>- Absolvieren von Challenges</li> <li>- Erhalten von Punkten</li> <li>- Einlösen von Punkten gegen Investition in Aufforstungen o.Ä.</li> <li>- Artikel lesen und kommentieren</li> </ul>                                                                                                                                                                                               | <ul style="list-style-type: none"> <li>- Investition von Punkten in Aufforstungen o.Ä.</li> </ul>                                                                                                                                                                  | <ul style="list-style-type: none"> <li>- Rankings (Families / Communities), Level, Artikel, Events, Forum, Petitionen, wöchentliches Ziel, Streak fehlen</li> <li>- Keine genaue Quantifizierung der Aktionen</li> </ul>                                                                                                                                                      |
| <b>EARTH HERO</b>  | <ul style="list-style-type: none"> <li>- Einmaliges Eintragen allgemeiner Aktionen</li> </ul>                                                                                                                                                                                                                                                                                                                                  | <ul style="list-style-type: none"> <li>- Erhalten von Klimapunkten / Klimatalern für klimafreundliche Aktivitäten</li> <li>- Level</li> </ul>                                                                                                                      | <ul style="list-style-type: none"> <li>- Rankings (Families / Communities), Artikel, Events, Forum, Petitionen, wöchentliches Ziel, Streak fehlen</li> <li>- Keine genaue Quantifizierung der Aktionen</li> </ul>                                                                                                                                                             |

Tabelle 1: Vergleich verschiedener Nachhaltigkeitsapps mit ClimateQuest

Diese Apps verfolgen, genau wie meine App, das Ziel, Nutzerinnen und Nutzer zu mehr Klimaschutz zu bewegen. Allerdings fehlen bei vielen Apps aus meiner Sicht wichtige Features, die meine eigene App haben soll, vor allem das System der Families und Communities und die Möglichkeit, eigene Artikel, Events, Forumsbeiträge und Petitionen hinzuzufügen. Daher ist die Programmierung dieser App keinesfalls redundant, sondern möglicherweise ein Schritt in Richtung mehr Klimaschutz im Alltag.

### 3. Vorüberlegungen

#### 3.1. Kooperationen

Um fachliche Expertise zum Inhalt der App beziehungsweise den klimafreundlichen Aktionen, welche man eintragen kann, zu erhalten, habe ich ausgewiesene Experten/innen im Bereich Klimaschutz angeschrieben, und diese um Tipps zur Recherche sowie Feedback gebeten. Während einige der angefragten Personen bzw. Institutionen wie beispielsweise Prof. Dr. Pongratz vom Münchner Zentrum für Nachhaltigkeit oder der Lehrstuhl für Umwelt- und Klimapolitik der TUM aus zeitlichen Gründen keine Unterstützung leisten konnten oder nicht antworteten, konnte ich dennoch mit den folgenden Personen Gespräche führen, in denen ich konkrete Hinweise, fachliches Feedback und hilfreiche Anregungen für die Weiterentwicklung der App erhalten habe:

- **Herr Prof. Dr. Dr. Höppe<sup>2</sup>**, ehemaliger Bereichsleiter Geo Risk Research/Corporate Climate Centre, Munich Re Experte für Naturgefahren, Klimawandel und Biometeorologie und Vorsitzender der Münchener Universitätsgesellschaft, hat mir ein persönliches, telefonisches Gespräch angeboten, in welchem er mir Tipps zum generellen Aufbau und Umfang der Aktionen und einige hilfreiche Internetseiten mitteilte. Darüber hinaus bot er mir an, meine fertige schriftliche Hausarbeit Korrektur zu lesen. Zehn Tage nach unserem Gespräch sendete er mir eine E-Mail, in welcher er mir schrieb, dass er einem Bekannten, Dr. Weibl, Gründer der Firma Telusio, welche den CO<sub>2</sub>-Fußabdruck verschiedener Produkte quantifiziert, von meinem Projekt erzählt habe und ich mich für weitere Tipps an diesen wenden könne. Nach einem bezüglich der App arrangiertem, persönlichen Treffen berichtete er auf seinem gut frequentierten LinkedIn Profil von meinem Projekt, was auch zahlreiche Likes, zum Beispiel vom Vizepräsidenten der LMU, generierte. Zudem bot Prof. Dr. Dr. Höppe mir an, dass ich meine App bei einer seiner Vorlesung an der Universität im Mai vorstellen dürfe.
- Nachdem ich auf den Rat von Herrn Prof. Dr. Dr. Höppe auch **Herrn Dr. Weibl** angeschrieben hatte, bot mir dieser ebenfalls ein persönliches Gespräch via Microsoft Teams an. In diesem nannte er mir eine Webseite, welche die CO<sub>2</sub>-Emissionen verschiedener Werkstoffe und Prozesse quantifiziert. Zudem gab er mir Tipps zum Heraussuchen der CO<sub>2</sub>-Emissionen der Aktionen und bot mir weitere Unterstützung an.

Seit dem Regionalwettbewerb am 26.02. konnte ich einen zusätzlichen Unterstützer meines Projekts gewinnen: Ich habe nämlich inzwischen eine Funktion entwickelt, über die Nutzerinnen und Nutzer durch gesammelte Klimapunkte sogenannte Credits erhalten können, mit denen reale Baumpflanzungen finanziert werden sollen, um digitale Klimaschutzaktionen direkt mit realen Umweltmaßnahmen zu verknüpfen. Diese Funktion befindet sich derzeit allerdings noch im Entwicklungsmodus und ist aktuell nur im Entwicklungsmodus zugänglich, was mithilfe einer eigenen Bibliothek umgesetzt wurde. Um die Baumpflanzungen zu realisieren, habe ich *Plant-for-the-Planet* angeschrieben, eine Organisation, die Bäume pflanzt, um dem Klima zu helfen, und nachgefragt, inwieweit eine Kooperation möglich wäre. In einem Video-Call mit Herrn Karim Kaamel, zuständig für Unternehmenspartnerschaften bei *Plant-for-the-Planet*, wurde ich auch die Möglichkeit hingewiesen, durch eine API Bäume pflanzen zu lassen. Auf seinen Ratschlag hin werde ich bestimmte Unternehmen anzuschreiben, die diese Baumpflanz-Aktionen ggf. finanzieren könnten, und darf in meinen Anschreiben auf die Unterstützung durch

---

<sup>2</sup> LMU München, Themengebiet – LMU München, <https://www.lmu.de/de/die-lmu/struktur/zentrale-universitaetsverwaltung/presse-und-kommunikation-puk/expertenservice/themengebiet/wetter.html>, zuletzt aufgerufen: 29.12.2025

durch *Plant-for-the-Planet* aufmerksam machen. Außerdem habe ich das Angebot von Herrn Kaamel, meine App durch das Social-Media-Team von *Plant-for-the-Planet* promoten zu lassen.

### 3.2. Aktionen

Um eine klare Skala zum Quantifizieren der Klimapunkte zu erhalten, gibt es meiner Meinung nach zwei Optionen: Die eingesparten CO<sub>2</sub>-Äquivalente oder den ökologischen Fußabdruck. Wenngleich bei Letzterem auch der Wasserverbrauch, der Einsatz von Pestiziden, der Schutz von Ökosystemen o.Ä. mit einberechnet wird, habe ich mich dennoch für die erste Möglichkeit entschieden, da man zum ökologischen Fußabdruck viel weniger Literatur findet und manche Indikatoren des ökologischen Fußabdrucks den Klimawandel nicht direkt beeinflussen.

Die Datenbankgrundlage, also die Aktionen, welche eingetragen werden können, muss auf vertrauenswürdigen Quellen basieren. Hierfür habe ich eine Internetrecherche durchgeführt, allerdings nur eine wirklich gute Quelle gefunden, die für mehrere Aktionen angibt, wie viel CO<sub>2</sub> man sparen kann, wenn man diese ausführt. Das Klimasparbüchle, herausgegeben von „Geschäftsstelle der Nachhaltigkeitsstrategie Ministerium für Umwelt, Klima und Energiewirtschaft Baden-Württemberg“, zeigt, wie viel Kilogramm CO<sub>2</sub> jährlich durch verschiedene Aktionen eingespart werden können.<sup>3</sup> Außerdem habe ich meiner Datenbank noch andere Aktionen hinzugefügt, deren CO<sub>2</sub>-Ersparnis ich durch andere, nur auf diese Aktion zugeschnittene Quellen recherchiert habe.

Bei zeitlichen Aktionen, wie beispielsweise vegetarischer Ernährung, habe ich als Einheit Wochen verwendet, damit User tatsächlich ein greifbares, in der Nähe liegendes Ziel haben, an dem sie sich bei der Durchführung ihrer Aktion orientieren können, welches weder zu anspruchsvoll noch zu einfach ist. Zudem wären längere Einheiten nicht gut mit dem wöchentlichen Ziel vereinbar, da User ansonsten möglicherweise Aktionen durchführen, aber noch nicht eingetragen haben und das wöchentliche Ziel daher nicht erreichen.

Aktionen, die negative Klimapunkte bringen, also beispielsweise Flugreisen, habe ich bewusst nicht hinzugefügt, da hier die meisten Menschen vermutlich nichts eintragen würden und Ehrlichkeit so bestraft würde. Die App soll vielmehr auf dem Belohnungsprinzip basieren. So erhält man in diesem Beispiel zwar keine Minuspunkte, wenn man Flugzeug fliegt, wohl aber Pluspunkte, wenn man nicht fliegt.

Um Aktionen leichter zu finden, habe ich diese in Kategorien wie Ernährung, Konsum oder Mobilität eingeteilt. Möchte man eine Aktion eintragen, so wählt man zuerst eine Kategorie und anschließend eine Aktion in dieser Kategorie aus. Alle von mir in die App aufgenommenen Aktionen sind inklusive ihrer Quellen auf <https://climate-quest.de> einsehbar. Da die Aktionen dynamisch aus einer Datenbank geladen werden, ist diese Liste jederzeit erweiter- oder bearbeitbar.

Damit keine allzu unrealistischen Angaben gemacht werden können, werden die eingetragenen Aktionen im Hintergrund überprüft: Hat man insgesamt bei einer Art von Aktion mehr als ein vorgegebenes Maximum als Menge eingetragen – als Maximalwert habe ich eigentlich unmöglich zu erreichende Werte verwendet – wird das Eintragen der Aktion blockiert. Zudem dürfen zeitliche Aktionen weder zweimal im selben Zeitraum eingetragen werden noch vor mehr als einem Monat begonnen haben. Dies sorgt auch dafür, dass Aktionen, die schon vor

---

<sup>3</sup> Klimasparbüchle, Geschäftsstelle der Nachhaltigkeitsstrategie, Ministerium für Umwelt, Klima und Energiewirtschaft, Baden-Württemberg ([https://um.baden-wuerttemberg.de/fileadmin/redaktion/m-um/intern/Dateien/Dokumente/2\\_Presse\\_und\\_Service/Publikationen/Klima/Klima-Sparbuechle-barrierefrei.pdf](https://um.baden-wuerttemberg.de/fileadmin/redaktion/m-um/intern/Dateien/Dokumente/2_Presse_und_Service/Publikationen/Klima/Klima-Sparbuechle-barrierefrei.pdf)), zuletzt aufgerufen: 08.12.2025



langer Zeit durchgeführt wurden, nicht mehr eingetragen werden können, da die App es ja zum Ziel hat, zu neuen nachhaltigen Aktionen zu motivieren.

### 3.3. Design

Das Design ist ein wichtiger Bestandteil jeder Webseite, da dies, zusammen mit der Performance bzw. Ladegeschwindigkeit, der erste Eindruck ist, den ein User von einer Webseite erhält.

Für eine konsistente Farbgebung habe ich CSS-Variablen verwendet, welche die Hauptfarben einheitlich angeben. Für Desktops nutze ich die Navigationsleiste mit Dropdown Menu, die sowohl über Klicks als auch durch Hovern verfügbar ist. Für kleinere Bildschirme habe ich zuerst ein Hamburger-Menü verwendet, bei welchem die Navigationsleiste seitlich eingefahren wurde. Später bin ich allerdings auf ein klassisches Mobile-App-Menü umgestiegen, um bei PWAs (siehe 5.8. Mobile Apps) ein natives Gefühl zu ermöglichen. Dadurch sind im unteren Bereich die verschiedenen Tabs durch Icons verfügbar. Dieses Design hat mir so gut gefallen, dass es nun auch bei Browsern auf kleineren Geräten statt des Hamburger-Menüs angezeigt wird. Meiner Meinung nach kann man die App auch sehr gut auf kleineren Bildschirmen (Smartphones, Tablets) verwenden, was auch die SEO (siehe 5.4. SEO) verbessert.

Eine weitere Herausforderung war die Anordnung der „fixed-Elements“, die immer an der gleichen Stelle angezeigt werden. Dazu gehören beispielsweise der Up- und der Cookie-Reset-Button sowie bei mobilen Apps die Browser-Elemente. Bei den Family- und Community-Rankings gibt es außerdem einen fest positionierten „Zu mir“-Button. Damit sich keine Elemente überlappen oder, bei kleineren Bildschirmen, über bzw. unter der ebenfalls fixierten Navigationsleiste stehen, muss eine geschickte Anordnung der Elemente gewährleistet sein. Daher entschied ich mich dafür, die drei Buttons in den Ecken zu platzieren. Die Browser-Elemente können ein- und ausgeklappt werden, damit sie keine Überschriften oder anderen Content verdecken, und befinden sich dann in der linken oberen Ecke. Damit die unteren Elemente auf kleineren Bildschirmen nicht durch die Navigationsleiste verdeckt werden oder diese verdecken, aber bei größeren Bildschirmen auch nicht zu weit im Bild stehen, habe ich als Abstand vom bottom zu der Höhe der Navigationsleiste im responsiven Modus immer 5px hinzugefügt, ansonsten aber 1rem (rem bezieht sich auf die Schriftgröße im root, also dem gesamten html) genommen.

Um ein moderneres Design zu ermöglichen, habe ich Hover-Effekte hinzugefügt, sodass sich beispielsweise die Farbe von Buttons ändert, wenn man mit der Maus über diese hovers. Dies hat allerdings das Design auf dem Handy verschlechtert, da es hier keine Maus gibt: Hat man auf einen Button gedrückt, so bleibt dieser im Hover-Zustand. Daher habe ich mit JavaScript ein Skript geschrieben, welches, sofern das Gerät nicht hoverfähig ist, die Hover-Effekte der Seite deaktiviert.

## 4. Programmierung

### 4.1. Allgemeines

Meine Web-App *ClimateQuest* habe ich mit Django, einem Framework zur Backend-Entwicklung in Python, programmiert. Dieses liefert bereits viele Funktionen mit wie z.B. den E-Mail-Versand, Token-Generierung, Passwort-Hashing oder ein Timezone-Modul. Das Frontend habe ich mit HTML, um Inhalte anzuzeigen, CSS zum

Design und JavaScript, um dynamische Frontend-Funktionen wie Passwort-Empfehlungen, einen Up-Button oder das Fetchen von Daten bereitzustellen, programmiert. Den grundlegenden Inhalt, der auf jeder Seite zu sehen ist, wie beispielsweise die Navigationsleiste, habe ich mithilfe einer base.html erstellt. Alle anderen HTML-Dateien erben von dieser Datei und setzen einige Parameter in den <head>, <title> oder <main> Tag hinein. Auch das ist eine von Django mitgelieferte Funktion. Der gesamte Code kann unter <https://github.com/Hipposmile/ClimateQuest> eingesehen werden.

## 4.2. Funktionen der Webseite

- **Registrierung, E-Mail-Adressen Verifizierung Login, Passwort-Reset:** Dies sind die grundlegenden Funktionen der meisten Apps. Ich nutze hierbei das von Django mitgelieferte Authentication-Tool. Da ich persönlich Webseiten, bei denen man seine E-Mail Adresse angeben muss, aus Datenschutzgründen oft wegklicke, habe ich mich dazu entschieden, dass User die E-Mail Adresse bei der Registrierung angeben können, aber nicht müssen. Ist eine E-Mail Adresse angegeben, erhält man zusätzliche Funktionen wie das Passwort-Reset. Um die E-Mail-Adresse zu verifizieren, habe ich als erstes eine neue Datenbank hinzugefügt, um VerifizierungsCodes zu speichern, welche anschließend 24 Stunden gültig sind. Später habe ich das Konzept allerdings überarbeitet, sodass mittlerweile ein Verifizierungslink, bestehend aus der UUID, also der sehr langen und fast unmöglich zu erratenden ID des Users, sowie einem Token gesendet wird, der beim Anklicken die E-Mail-Adresse verifiziert. Um zu verhindern, dass Bots sich registrieren können, habe ich bei der Registrierung außerdem reCAPTCHA implementiert, ein Tool von google, mit welchem auf Basis des Userverhaltens auf der Webseite entschieden wird, ob es sich vermutlich um einen Bot handelt und dementsprechend eine Registrierung erfolgen darf oder nicht. Ich habe mir selbst vorgegeben, dass sowohl Username als auch E-Mail-Adresse unique sein müssen, um den Benutzer auch vor anderen Usern eindeutig zu identifizieren. Die E-Mail-Adresse muss zum Passwort-Reset eindeutig sein. Dies ist nicht ganz einfach, da das User-Model nicht bearbeitet werden kann, denn es ist von Django vorgegeben. Zwar wird bei der Registrierung darauf geachtet, dass es einen Usernamen nicht doppelt gibt, sollte es hier allerdings Probleme geben, würde kein Fehler ausgegeben, sondern es würden die Funktionalität beeinträchtigt. Ferner gibt es keine Möglichkeiten, zusätzliche Werte zu den Usern zu speichern, z.B., ob sie Mails erhalten wollen oder die E-Mail Adresse bereits verifiziert ist. Daher habe ich ein zweites Modell – UserErweitert – hinzugefügt, um hierin sowohl den zugehörigen User als auch diese Werte zu speichern.
- **Erstellen von Aktionen und Erhalten von Klimapunkten und Leveln:** User können Aktionen eintragen und dafür Klimapunkte erhalten. Im Backend gibt es dafür eine Tabelle in der Datenbank mit allen Aktionen, die man eintragen kann. Erstellt ein User eine Aktion, so wird in einer anderen Tabelle ein Eintrag erstellt, in dem die eingegebene Menge, die optionale Beschreibung, das angegebene Datum, an dem die Aktion ausgeführt wurde, der User, der die Aktion eingetragen hat, und ein sogenannter ForeignKey, der auf die Aktion verweist, hinterlegt sind. Durch Multiplikation der vom User eingegebenen Menge und der für die jeweiligen Aktionen bereits vorgegebenen Klimapunkte, welcher der User pro Einheit für die Aktion erhält, werden die Klimapunkte berechnet, die der User für das Eintragen erhält. Dies passiert jedesmal von Neuem, was zwar nicht sehr effizient ist, aber nicht nur dafür sorgt, dass Änderungen an der Datenbankgrundlage auch bei bereits eingetragenen Aktionen wirken, sondern auch Speicherplatz spart und den Code sowie die Datenbank übersichtlicher macht. Alle

Levelbezeichnungen inklusive der hierfür notwendigen Klimapunkte sind wiederum in einer eigenen Tabelle gespeichert. Bei jedem Aufruf wird das Level des Users neu berechnet.

- **Erstellen, Beitreten, Einsehen von Families:** In einer Family können sich mehrere User zusammenschließen und vergleichen. Mithilfe eines Admin-Passworts können die Daten wie Name, Passwort, Admin-Passwort oder Mitglieder der Family bearbeitet werden. Außerdem gibt es eine Chat-Funktion, in die jedes Mitglied der Family Nachrichten schreiben kann, z.B. Vorschläge, wie alle zusammen eine klimafreundliche Aktion durchführen könnten. In den Einstellungen kann man auswählen, ob diese Chat-Funktion verfügbar ist oder nicht. Darüber hinaus gibt es eine bereits vorgefertigte Family namens „worldwide ranking“, in der alle User Mitglied sind. Bei der Registrierung wird man dieser Family automatisch hinzugefügt.
- **Erstellen, Beitreten, Einsehen von Communities:** Mehrere Families können sich zu einer Community zusammenschließen. Der Aufbau ist sehr ähnlich zu den Families; das Ranking mit den Klimapunkten, die Bearbeitungs- und Chatfunktionen sind identisch. Der Vorteil von Communities gegenüber Families wird im Projektüberblick erläutert.
- **Erstellen von Artikeln:** In Artikeln kann man sein eigenes Wissen zu Klimaschutzthemen mit anderen teilen. Andere User können über die Kommentarfunktion Fragen und Anregungen zu dem Artikel hinterlassen; auch das Liken des Artikels ist möglich. *Daum runter* gibt es nicht. Für jeden Like erhält der Artikelautor 0,1 Klimapunkte. Wenn man also sehr beliebte Artikel schreibt, kann man auch viele Klimapunkte erhalten. Artikel des Users *admin* sind als offizielle Artikel von *ClimateQuest* gekennzeichnet. Um Fake-News zu verhindern, kann ich als Admin alle Artikel, die ich auf FakeNews überprüft habe, verifizieren bzw. sperren. Gesperrte Artikel können trotzdem eingesehen werden, User erhalten nur eine Nachricht, die erklärt, dass der Artikel falsch ist. Um Designaspekte wie fetten, kursiven oder unterstrichenen Text, Links und Stichpunkte einzufügen, habe ich Quill, einen sogenannten Rich-Text-Editor, eingebunden. Quill bietet eine grafische Oberfläche, mit der man eingegebenen Text formatieren kann. Diese Eingabe wird dann in HTML-Code umgewandelt und im Backend gespeichert.<sup>4</sup> Auf dieselbe Weise können auch Events und Forumsbeiträge gestylt werden.
- **Wöchentliches Ziel:** In der Datenbank wird für jeden User ein wöchentliches Ziel gespeichert, welches vom User bearbeitet werden kann. Zudem werden alle in der letzten Woche gesammelten Klimapunkte addiert und daraus der Fortschritt auf dem Weg zum eigenem individuellen Klimaziel berechnet.
- **Streak:** Um die Streak zu berechnen, werden alle Datumsangaben der Aktionen des Users gespeichert und von der aktuellen Woche ausgehend auf Wochen geprüft, in denen das Ziel nicht erreicht wurde. Wenn in der aktuellen Woche das Ziel noch nicht erreicht wurde, wird die Streak ausgehend von der letzten Woche angezeigt.
- **Tracking:** Alle Aktionen, die jede Woche automatisch eingetragen werden sollen, werden in einer eigenen Datenbank-Tabelle gespeichert. Ein Cronjob trägt dann jeden Sonntag alle dieser Aktionen automatisch ein. Für das GPS-Tracking wird mithilfe der JavaScript-Funktion *watchPosition* der Standort überwacht. Bei Änderungen wird die Distanz zwischen dem alten und dem aktuellen Standort berechnet und zur insgesamt bereits getrackten Distanz addiert. Die Standortdaten werden nur im

---

<sup>4</sup> Auf welche Weise hierbei auf Hacksicherheit geachtet wird, ist im Abschnitt 5.7. Hackersicherheit beschrieben.

Frontend behandelt, erst wenn auf *Stoppen* gedrückt wird, sendet das Frontend die Distanz ans Backend. Der Standort wird also nicht dauerhaft gespeichert.

### 4.3. Probleme beim Programmieren

Die Programmierung dieser Webseite war mit vielen kleinen und größeren Bugs verbunden, wie so oft beim Coding. Die meisten davon waren nur Tipp- oder Logikfehler, die sich recht schnell beheben ließen. Einige der größeren Fehler sind im Folgenden exemplarisch aufgeführt:

- **E-Mail-Versand:** Der E-Mail-Versand hat mir lange Probleme bereitet, denn beim Ausführen auf dem Localhost kam immer wieder die Fehlermeldung: *[SSL: CERTIFICATE\_VERIFY\_FAILED] certificate verify failed: self-signed certificate in certificate chain (\_ssl.c:1028)*, zu dessen Lösung ich im Internet nichts finden konnte. Nach einiger Zeit begann ich zu vermuten, dass dies mit dem localhost zu tun hat, was sich bei bzw. nach der Veröffentlichung bestätigt hat, denn auf dem nginx/gunicorn-Server funktioniert der E-Mail-Versand problemlos. Dies sorgt allerdings auch dafür, dass ich Funktionen, die mit dem E-Mail-Versand zusammenhängen, nicht lokal testen kann, sondern nur auf dem Server. Am Anfang habe ich die E-Mails zum Testen in das Terminal printen lassen, mittlerweile geht das nicht mehr, da sonst auch im Produktionsmodus die E-Mails in den Logs und nicht in den E-Mail-Postfächern der User landen würden. Daher gab es vor allem bei der E-Mail-Adressen-Verifizierung und beim Passwort-Reset einige Startschwierigkeiten, welche sich allerdings schnell und durch nur kleine Codeänderungen lösen ließen.
- **Deutsche Schreibweise von Zahlen:** Da Dezimalzahlen im Englischen mit einem Punkt und im Deutschen mit einem Komma getrennt werden, hat mich das deutsche Design vor Schwierigkeiten gestellt. Sowohl JavaScript als auch Python sind englische Programmiersprachen und trennen deshalb Dezimalzahlen mit einem Punkt. Im Frontend soll allerdings die deutsche Schreibweise mit einem Komma angezeigt werden. Glücklicherweise gibt es in Django die Möglichkeit, den Stil der Zahlen, die im Frontend angezeigt werden, zu verändern, sodass Zahlen automatisch in der deutschen Schreibweise angezeigt werden. Komplexer wird es, wenn im Frontend Dinge mithilfe von JavaScript berechnet werden: Die Grundlage dieser Berechnungen sind meist deutsche Zahlen, weshalb als erstes eine Art Compiler, der deutsche Zahlen ins Englische umwandelt, geschrieben werden musste. Zum Anzeigen der Berechnungen ist genau das Gegenteil notwendig, auch bei Formularen, bei denen deutsche Dezimalzahlen angegeben werden, muss vor dem Absenden des Formulars dieser Input in eine englische Zahl umgewandelt werden, bevor Python hiermit weiterrechnen kann.

## 5. Veröffentlichung

### 5.1. Schritte bei der Veröffentlichung

Um die App zu veröffentlichen, habe ich einen Debian VPS (=Virtual Private Server) mit folgenden Tools eingerichtet: Gunicorn, ein WSGI-Server, der Django ausführt, Supervisor, ein Tool, das Gunicorn überwacht und ggf. neustartet, sowie nginx, ein Webserver, der alle http(s)-Anfragen entgegennimmt, die statischen Dateien wie Bilder, CSS oder JavaScript ausliefert (mittlerweile übernimmt das ServeStatic, siehe 5.5. Performance) und das SSL-Zertifikat einbindet. Als Datenbankmanagementsystem habe ich PostgreSQL verwendet, zur Übertragung des

Codes von meinem lokalen Rechner auf den VPS GitHub. Das SSL-Zertifikat habe ich mit Certbot bzw. Let's Encrypt erstellt und eingebunden, zudem habe ich einen DNS-Forward von meiner Domain [https://\(www.\)climate-quest.de/](https://(www.)climate-quest.de/) auf meinen VPS erstellt und zur Effizienzsteigerung einige Skripte geschrieben, welche meinen Code beispielsweise automatisch von meinem lokalen Rechner auf GitHub und den VPS laden. Zudem habe ich Cronjobs hinzugefügt, die täglich Backups der Datenbank machen, die E-Mail-Adresse [kontakt@climate-quest.de](mailto:kontakt@climate-quest.de) über einen externen Mailserver eingerichtet und mir einen Trustpilot Business Account angelegt, damit ich Bewertungen und Feedback erhalten kann.

## 5.2. Probleme bei der Veröffentlichung

Bei der Veröffentlichung gab es, wie beim Coding, Probleme. Hier sollen lediglich einige der größeren Schwierigkeiten exemplarisch kurz dargelegt werden.

- **500 Bad Gateway:** Dieser Fehler wurde ausgegeben, wenn in den Settings von Django in den Allowed Hosts nicht die Domain angegeben war. Auch bei der Subdomain www ist dies notwendig. Dieser Fehler ließ sich allerdings durch das Aktualisieren der Django-Settings recht schnell beheben.
- **Erzwungene Datenbank-Resets:** Teilweise ist es zu erzwungenen Resets meiner Datenbank gekommen, da Codefehler ganze Datenbanktabellen gelöscht haben und unregelmäßige Migrationsdateien das Aktualisieren der Datenbankstruktur unmöglich gemacht haben
- **Fehler bei www und no-www:** Damit meine Seite über [climate-quest.de](https://climate-quest.de) wie auch über [www.climate-quest.de](https://www.climate-quest.de) verfügbar ist, musste ich sowohl für den Host @ als auch für www DNS-Einträge tätigen und in den settings\_prod sowohl bei ALLOWED\_HOSTS, bei INTERNAL\_IPS als auch bei CSRF\_TRUSTED\_ORIGINS nicht nur [climate-quest.de](https://climate-quest.de), sondern auch [www.climate-quest.de](https://www.climate-quest.de) angeben. Daher habe ich das SSL-Zertifikat und die nginx-Konfiguration für beide Seiten trotz gleichen Inhalts erstellt.

## 5.3. Rechtliches

Schon bei der Programmierung habe ich sehr auf DSGVO (= Datenschutz-Grundverordnung-)Konformität geachtet. Jedes externe Skript, das eingebunden wird, erfordert eine Cookie-Zustimmung, sonst wird es nicht geladen. Hierfür habe ich eine JavaScript-Datei erstellt, welche in base.html eingebunden wird. Dadurch wird ein Cookie-Banner eingefügt, bei dem man *Alle ablehnen*, *Alle akzeptieren* oder *Benutzerdefinierte Einstellungen* auswählen kann. Diese Einstellungen werden im LocalStorage gespeichert, sodass sie auch beim nächsten Besuch der Webseite oder beim Neuladen vorhanden sind. Ein weiterer Teil des Skripts sucht automatisch nach allen divs, also HTML-Containern, die die Klasse *cookie-blocked* besitzen. Das Skript zeigt dann alles an, was in *data-content* steht, sofern *data-requires* zugestimmt wurde, ansonsten ersetzt er es mit *Dieses Element ist blockiert. Bitte '...'-Cookies erlauben, um es zu sehen. Evtl. musst du die Seite anschließend neu laden*. Skripte, die in *data-content* eingebunden wurden, werden automatisch geladen. Außerdem kann von anderen Stellen im Code auf die von mir definierte Funktion *getCookiePreferences()* zugegriffen werden, die einem sagt, ob einer bestimmten Art von Cookies zugestimmt wurde. So wird gewährleistet, dass externe Skripte nur geladen werden, wenn den entsprechenden Cookies auch zugestimmt wurde.

Auch habe ich auf WCAG (Web Content Accessibility Guidelines) bzw. Barrierefreiheit geachtet, sodass beispielsweise Screenreader den Inhalt gut wiedergeben können und die Kontraste zwischen Text und Hintergrund

hoch sind. Zur Überprüfung habe ich verschiedene Tools wie Google Lighthouse, wave.webaim.org oder das Firefox Plugin Accessibility Assessment verwendet.

Meine Website verfügt ferner über eine Datenschutzerklärung, Nutzungsbedingungen und ein Impressum. Diese ließ ich von einem Anwalts-ChatGPT, welches ich von einem mit mir verwandten Juristen bekommen habe, darauf prüfen, ob diese rechtlich angreifbar sind, und habe die von diesem ChatGPT genannten Probleme behoben.

Vor jedem neuen Eintrag und vor dem Hinterlassen oder Beantworten von Kommentaren muss bestätigt werden, dass dies der Wahrheit entspricht und mit den Nutzungsbedingungen übereinstimmt, um Ehrlichkeit sicherzustellen, auch wenn dadurch kein sicherer Schutz gewährleistet wird. Um diesen zu verstärken, gibt es die Möglichkeit, User zu melden, wenn diese gegen die Nutzungsbedingungen verstoßen.

## 5.4. SEO

Damit meine Webseite tatsächlich gefunden werden kann, habe ich SEO betrieben. SEO steht für Search Engine Optimization und sorgt dafür, dass Webseiten in den Suchergebnissen höher gelistet werden. Hierfür habe ich zuerst meta-Tags in den head-Bereich der Webseite eingefügt, welche vor allem für Suchmaschinen angeben, worum es auf der Webseite geht, oder sagen, ob die Webseite geindext werden soll. Außerdem habe ich eine sitemaps.xml mithilfe von Django erstellt, welche angibt, wie häufig eine Seite aktualisiert wird und wie wichtig sie ist – ein zentraler Aspekt für Suchmaschinen. Zusätzlich habe ich eine robots.txt erstellt, welche Suchmaschinen nicht nur genau sagt, welche Seiten geindext werden dürfen und welche nicht, sondern auch die sitemaps.xml verlinkt. Außerdem habe ich meine Webseite bei der Google Search Console registriert, welche Fehler bei der Webseitenkonfiguration und Indexierung ausgibt. Hier habe ich außerdem die sitemaps.xml eingetragen und die in der Search Console angegebenen Fehler so weit wie möglich behoben.

Außerdem habe ich meine Webseite bei Trustpilot registriert, damit User hier Feedback hinterlassen können und andere Menschen sich eine ehrliche Meinung über *ClimateQuest* bilden können. Auch dies verbessert die SEO, indem Backlinks von der vertrauenswürdigen Seite Trustpilot generiert werden. Zur Überprüfung wurde mit Google Lighthouse ein Test durchgeführt, welcher mir bei SEO 100% attestiert hat.<sup>5</sup> Außerdem kommt bei Google, Bing, Yahoo!, DuckDuckGo, Ecosia, Startpage und AOL unter dem Suchwort ClimateQuest meine Webseite auf Platz 1, obwohl es auch viele andere Webseiten mit dem gleichen Namen gibt. Auch die KIs Perplexity, ChatGPT, Gemini und Claude nennen meine App zuerst. Dass dies nicht nur bei von mir verwendeten Geräten der Fall ist, zeigt, dass dieses Ranking nichts mit meinem Suchverlauf zu tun hat.

## 5.5. Performance

Zur Performanceoptimierung wurde erneut Lighthouse verwendet und anschließend folgende Aspekte verbessert bzw. angepasst:

- **Cache-Lifetimes (django + nginx):** Damit statische Dateien wie Bilder oder CSS-Dateien nicht jedes Mal erneut geladen werden müssen, sondern aus dem Browser-Cache geholt werden, habe ich mithilfe

---

<sup>5</sup> Lighthouse ist ein Tool zum Bewerten und Überprüfen einer Webseite. Sie überprüft diese in den Kategorien Performance, Accessibility, Best Practices und SEO und gibt einen Score zwischen 1 und 100 in den jeweiligen Kategorien aus, außerdem werden die konkreten Fehlerquellen genannt.

der Bibliothek `ServeStatic` alle statischen Dateien im Format `name.hash.ende` - z.B. `style.031507e67139.css` - ausgeliefert. Das sorgt dafür, dass alle statischen Dateien einen Cache-Header bekommen, der sagt, dass der Browser nur jedes Jahr die Datei neu laden darf, wodurch die Ladezeiten verkürzt werden. Ändert sich die Datei, ändert sich auch der Hash und es wird eine neue Datei geladen, sodass man nicht mehr alte Dateien sieht, sondern immer die aktuellen. `ServeStatic` ersetzt `nginx` als Auslieferer, sodass `nginx` nur noch die `https`-Anfragen entgegennimmt und weiterleitet.

- **Font-Display:** Für Emojis verwende ich eine separate Schriftart, damit die Emojis auf allen Betriebssystemen gleichbleiben. Hierfür habe ich eine andere Schriftart in den statischen Dateien gespeichert. Wenn diese allerdings noch nicht vollständig geladen ist, wird nichts angezeigt, was die gefühlte Ladezeit erhöht. Deshalb habe ich zum `font-face swap` hinzugefügt, sodass die Emojis zuvor schon in der Standardschriftart angezeigt werden.
- **http/2 statt http/1:** `http/2` ist deutlich schneller als `http/1`, indem es eine erweiterte Komprimierungsmethode verwendet, Blockierungen verhindert und Ressourcen an einen Client pushen kann, bevor dieser danach fragt.<sup>6</sup> Dies lässt sich über `nginx` unkompliziert konfigurieren.
- **Images:** Da Bilder recht groß sind, habe ich meine `.png`- bzw. `.jpg`-Bilder in `.avif`- und `.webp`-Bilder umgewandelt, welche deutlich kleiner sind. Zudem habe ich diese in verschiedenen Größen bereitgestellt, sodass größere Bildschirme Bilder in höheren und kleinere Bildschirme in niedrigeren Auflösungen erhalten, da hier der Unterschied ohnehin unsichtbar ist.

## 5.6. Testen

Um alle Funktionen zu testen, habe ich mir ein Dokument erstellt, in welchem eingetragen werden kann, welche Features funktionieren, ob Fehlermeldungen, wenn beispielsweise die Nutzungsbedingungen nicht akzeptiert wurden, korrekt angezeigt werden, ob keine Serverfehler auftreten und ob es noch Verbesserungspotenzial gibt. Aufkommende Fehler habe ich gelöst und anschließend nochmals von Neuem begonnen, da Codeänderungen, die Bugs fixen, teilweise andere Funktionalitäten beeinflussen können.

Um die App auf Android zu testen, habe ich mir `Android Studio` heruntergeladen, um einen `Android Simulator` zu haben. Zudem habe ich `Lighthouse` zum Prüfen meiner Webseite verwendet (Erklärung siehe 5.4. SEO) und ein `AI-Test-Tool` verwendet, welche automatisiert auf Grundlage einer Beschreibung der Webseite alle Funktionen durchtesten soll. Dies hat allerdings nicht funktioniert.

Für eine möglichst objektive Einschätzung von Design, Userfreundlichkeit und Funktionalität zu erhalten, habe ich ein Online-Formular erstellt. Jeder kann dieses ausfüllen und mir mitteilen, was ihm an der Webseite gefällt und was noch verbessert werden kann. Zudem können User ein Feedback auf `Trustpilot` oder im `AppStore` hinterlassen.

Um die App in den `Google Play Store` zu laden, musste ein weiterer Test durchgeführt werden (siehe 5.8. Mobile Apps).

---

<sup>6</sup> Cloudflare, HTTP/2 vs. HTTP/1.1: Wie wirken sie sich auf die Web-Performance aus?, <https://www.cloudflare.com/de-de/learning/performance/http2-vs-http1.1/>, zuletzt aufgerufen: 25.12.2025

## 5.7. Hackersicherheit

Ein zentraler Aspekt bei der Erstellung einer Website ist die Sicherheit: Sowohl der Schutz von Daten als auch die Verhinderung unbefugter Zugriffe und die Veränderung des Systems oder des Interfaces müssen gewährleistet sein. Django bringt bereits standardmäßig viele Tools hierfür mit, sodass XSS-Attacken, SQL-Injections oder CSRF-Attacken<sup>7</sup> praktisch unmöglich gemacht werden, sofern der Entwickler diese Regeln nicht lockert. Manchmal ist dies allerdings notwendig, da beispielsweise die Artikel HTML-Code beinhalten, der auch angezeigt werden soll, um bestimmte Designfunktionen wie z.B. fetten Text zu ermöglichen. Um Hacking zu verhindern, wird der Input vorher mithilfe der Python-Bibliothek bleach auf schädlichen HTML-Code untersucht, bzw. werden nur ausgewählte HTML-Elemente erlaubt. Bei allen anderen Elementen werden für HTML-Code unerlässliche Zeichen wie `<` oder `>` durch Unicode-Symbole ersetzt, sodass JavaScript Code wie `<script>alert("Hello World");</script>` nicht ausgeführt, sondern genauso angezeigt wird. Dies wurde von mir ausgiebig und erfolgreich getestet.

Damit Spam verhindert wird, wurde reCAPTCHA bei der Registrierung eingefügt, sodass sich keine Bots registrieren können (siehe 4.1. Allgemeines). Zudem bietet der Hoster meines VPS bereits einen vorinstallierten Schutz vor DDoS-Attacken an. Da ich meinem VPS eine Firewall hinzugefügt habe, ist dieser auch vor unautorisiertem Zugriff geschützt.

Zudem habe ich ein SSL-Zertifikat zu meiner Webseite hinzugefügt (siehe 5.1. Schritte bei der Veröffentlichung), welches die Kommunikation zwischen Client und Server verschlüsselt. Dieses habe ich mithilfe von Wireshark, einem Tool, mit dem man die requests zwischen Clients und Servern im eigenen W-LAN mitlesen kann, getestet.

Außerdem habe ich beim Schreiben des Codes bzw. beim Auslesen der Formulare, die User ausfüllen, darauf geachtet, dass, falls der Frontend-Code geändert wird, das Programm weder abstürzt noch falsche Daten speichert, sondern eine Fehlermeldung ausgibt.

Des Weiteren habe ich 10 http-Header zu meiner Webseite hinzugefügt, die bestimmte Angriffe verhindern, indem sie beispielsweise die Einbindung der Webseite in fremde iFrames oder verhindern oder Browser zwingen, https zu verwenden, auch wenn explizit http eingegeben wird. Besonders anspruchsvoll war hierbei die Content Security Policy, die alle Skripte oder Styles blockiert, wenn sie nicht explizit mithilfe dynamischer *nonces* als vertrauenswürdig markiert wurden und so XSS-Attacken verhindert, selbst wenn das Backend einen Fehler macht und gefährlichen Code nicht escapet oder bleached.

Um die Webseite auf Sicherheitslücken zu prüfen, habe ich sie mit OWASP ZAP<sup>8</sup> und SonarQube<sup>9</sup> gescannt und vorhandene sowie relevante Sicherheits- oder Qualitätsdefizite behoben.

## 5.8. Mobile Apps

Da es sich bei meiner App um eine Web-App handelt, kann sie plattformübergreifend verwendet werden. Allerdings ist es für User oft einfacher, solche Apps als reale Apps auf dem Smartphone zu besitzen. Daher wäre die Entwicklung einer kompatiblen iOS- und Android-App zielführend. Das geht beispielsweise mit dem Django

---

<sup>7</sup> Wallarm, Was ist ein CSRF-Token? Bedeutung und Beispiele, <https://lab.wallarm.com/what/csrf-token/?lang=de>, zuletzt aufgerufen: 01.03.2026

<sup>8</sup> ZAP, <https://www.zaproxy.org/>, zuletzt aufgerufen: 01.03.2026

<sup>9</sup> Sonar, SonarQube: Bekämpfen Sie AI Slop und überprüfen Sie AI-Code, <https://www.sonarsource.com/de/products/sonarqube/>, zuletzt aufgerufen: 01.03.2026



Rest Framework. Hierbei werden über eine API Daten abgerufen und anschließend in der App angezeigt. Dies könnte beispielsweise mit den Programmiersprachen Swift (iOS), Kotlin oder Java (Android) oder dem Framework Flutter (iOS und Android) gemacht werden. Da Flutter plattformunabhängig funktioniert, habe ich angefangen, eine mobile App mit diesem Framework zu bauen. Dann habe ich erfahren, dass man mit sogenannten PWAs (Progressive Web Apps) auch iOS- und Android-Apps erstellen kann. Hierbei handelt es sich um eine normale Webseite, die wie eine App wirkt, also beispielsweise auf dem Home-Screen installiert werden kann. Dies geht entweder über den Browser oder, mit weiterem Aufwand, auch über den App- oder Google Play Store.

Mit wenigen Änderungen, z.B. dem Hinzufügen eines `manifest.json` oder eines `service-worker.js`, kann aus einer Webseite eine PWA gemacht werden, welche sogar Push-Nachrichten senden kann. Folgende konkrete Änderungen habe ich durchgeführt, um aus meiner Webseite eine PWA zu machen:

- **manifest.json:** Das `manifest.json` gibt diverse Informationen wie den App-Namen, App-Icons, Shortcuts, die Theme-Color oder die Orientierung an.
- **sw.js:** Mit der ServiceWorker, einer JavaScript-Datei, werden beispielsweise Offline-Pages geladen, der Cache verwaltet oder Push-Nachrichten sendet.
- **Design:** Da in Browsern Back- und Forward-Buttons zur Navigation bereits vorinstalliert sind, musste ich, wenn sich die App im PWA-Modus befindet, noch manuell solche Buttons hinzufügen.
- **Push-Notifications:** Damit die PWA gegenüber der Webseite einen tatsächlichen Mehrwert bietet, habe ich mit `django-webpush` und dem Service-Worker die Möglichkeit, Push-Notifications zu empfangen, zu der App hinzugefügt. Nun werden die Benachrichtigungen an alle PWAs gesendet, in die der User eingeloggt ist. Bei Klick auf die Benachrichtigung öffnet sich das passende Fenster, beispielsweise der Chat einer Family, wenn die Nachricht anzeigt, dass man soeben eine neue Benachrichtigung erhalten hat. Diese Nachrichten funktioniert momentan allerdings nur, wenn die App als wirkliche PWA installiert wurde und nicht über den App / Play Store.
- **Browser-Elemente:** Zur leichteren Navigation habe ich Browser-Elemente hinzugefügt, die nur angezeigt werden, wenn die App als mobile App und nicht als Internetseite angezeigt wird. Um User nicht zu verwirren, wenn, anders als bei den Browsern, nicht angezeigt wird, dass die Seite lädt, habe ich einen Spinner hinzugefügt, der angezeigt wird, wenn eine Seite geladen oder ein Formular abgeschickt wird.

Um die App in den App Store zu bekommen, bietet der bereits erwähnte PWABuilder<sup>10</sup> eine Möglichkeit, einen Wrapper zu generieren, der die Webseite lädt. Lässt man diesen Wrapper erstellen, erhält man ein Package, das ohne weitere Bearbeitung dazu geeignet sind, in den App Store geladen zu werden.

Die Anforderungen, um eine PWA in den Apple App Store und den Google Play Store hochzuladen, sind verschieden. So müssen PWAs auch einen tatsächlichen Mehrwert gegenüber Webseiten haben, um in den Apple App Store aufgenommen zu werden.<sup>11</sup> Dies wäre beispielsweise durch Push-Nachrichten zumindest teilweise erreicht. Beim Google Play Store besteht diese Hürde nicht. Allerdings ist es hier so, dass vor der Veröffentlichung

---

<sup>10</sup> PWABuilder ist eine kostenlose Webseite, die einen dabei unterstützen kann, aus Webseiten PWAs zu machen.

<sup>11</sup> PWABuilder, Publish your PWA to the iOS App Store | PWA Builder Blog, <https://blog.pwabuilder.com/posts/publish-your-pwa-to-the-ios-app-store/>, zuletzt aufgerufen: 30.12.2025

durch private Entwicklerkonten Tests durchgeführt werden müssen, die mindestens 12 Leute für 14 Tage durchführen.<sup>12</sup> Für das Hochladen in den Apple App Store ist dies nicht notwendig.

Ich habe mir sowohl ein Google als auch ein Apple Entwicklerkonto angelegt und für beide Accounts neue *Apps* hinzugefügt. Anschließend habe ich jeweils den Store Eintrag konfiguriert, Fragebogen zum Bestimmen der Altersfreigabe ausgefüllt und das vom PWABuilder heruntergeladene Package nach leichter Anpassung hochgeladen. Bei Google wird nun momentan ein Test durchgeführt, Apple hat die App bereits überprüft. Zuerst wurde sie abgelehnt, was allerdings Fehlern in der Prüfung zu verdanken ist. Denn nach einer klärenden Antwort meinerseits wurde die App im App Store freigeschaltet und ist nun für alle zum Download unter <https://apps.apple.com/de/app/climatequest/id6759392266> verfügbar. Hierbei wurden auch schon Updates veröffentlicht, die beispielsweise für einen fließenden Übergang zwischen Browser und App sorgen oder Widgets einbinden, die ein schnelleres Eintragen von Aktionen ermöglichen.

## 6. Ergebnisdiskussion, Fazit, Ausblick

ClimateQuest ist bereits eine funktionierende Möglichkeit, sich selbst und andere zu klimafreundlichem Verhalten zu motivieren und hat dies auch bei ersten Nutzerinnen und Nutzern gezeigt. Für eine objektive Evaluation werden jedoch deutlich mehr Nutzende benötigt, die nicht aus meinem persönlichen Umfeld stammen und die App über einen längeren Zeitraum verwenden. Diese Kriterien sind momentan noch nicht erfüllt, weswegen eine Auswertung der App noch nicht möglich ist.

Dennoch habe ich bereits über mögliches Verbesserungspotenzial nachgedacht und bereits einige der beim Regionalwettbewerb noch hier aufgeführten Aspekte, wie das Verbessern der Hackersicherheit, die Veröffentlichung im App Store, das Prüfen der Webseite mit externen Tools oder die Einleitung erster Schritte für eine Kooperation mit *Plant-for-the-Planet* bezüglich Baumpflanzungen im Tausch gegen Klimapunkte, seit der Abgabe der ersten Version dieser Arbeit am 26.01.2026 realisiert.

Folgende Aspekte sollten weiterhin verbessert werden:

- **Refactoring:** Der aktuelle Code ist noch recht lang und unübersichtlich. Diesen im Sinne der Clean Code Regeln zu überarbeiten, würde der Lesbarkeit und Wartbarkeit gut tun.
- **Effizienz:** Die Webseite läuft aktuell noch nicht sehr effizient, da z.B. im Backend keine Caches verwendet werden. Indem man dies ändert, bei Rückverweisen von Models related\_name anstelle von unnötigen Datenbankabfragen nutzt und database access optimization<sup>13</sup> betreibt, kann die Effizienz der Webseite deutlich gesteigert werden. Das ist nicht nur für die Ladezeiten wichtig, sondern auch für die Nachhaltigkeit, da eine gesteigerte Effizienz auch den Stromverbrauch des Servers minimiert.
- **Automatisierte Tests:** Teilweise können einzelne Code-Änderungen Fehler an ganz anderen Stellen verursachen. Um nicht bei jeder Codeänderung überprüfen zu müssen, ob ein Fehler entsteht, und mehrere Stunden mit dem Testen beschäftigt zu sein, ist es praktikabel, automatisierte Tests zu schreiben, welche alle Funktion durchtesten. Django liefert hier bereits einige kleine Tools mit, die auf UnitTest, einer

---

<sup>12</sup> Play Console Hilfe, Anforderungen an App-Tests für neue private Entwicklerkonten - Play Console-Hilfe, <https://support.google.com/googleplay/android-developer/answer/14151465?hl=de>, zuletzt aufgerufen: 30.12.2025

<sup>13</sup> Django, Database access optimization | Django documentation | Django, <https://docs.djangoproject.com/en/6.0/topics/db/optimization/>, zuletzt aufgerufen: 29.12.2025

Python-Bibliothek, basieren. Hierbei kann man z.B. den Statuscode beim Aufrufen einer Webseite prüfen, ob bestimmte Teile auf der Webseite enthalten sind, ob die korrekten HTML-Files verwendet werden, ein HTML-File die korrekten Daten aus der Datenbank ausliest oder die Datenbank Daten überhaupt korrekt speichert.<sup>14</sup> Hierfür wären allerdings kleine Funktionen mit nur einem Aufgabenbereich notwendig, das heißt, das Refactoring müsste zuerst durchgeführt werden.

- **Funktionen:** Der Funktionsumfang der App ist schon recht umfangreich, doch einige zusätzliche Funktionen wären noch denkbar, wie beispielsweise eine Community, in der alle Families Mitglied sind; die Möglichkeit, Statistiken anzuzeigen, welche einem sagen, wann wie viele Klimapunkte in welchen Kategorien gesammelt wurden, welche Trends sich abzeichnen oder wo man sich am meisten verbessern kann oder eine Funktion, empfohlene Artikel, Forumsbeiträge und Events anzuzeigen, damit die Weiterbildung vereinfacht wird.
- **Internationalität:** Momentan ist meine App nur auf Deutsch verfügbar. Um eine möglichst große Reichweite zu erlangen, ist es notwendig, sie auch auf anderen Sprachen – insb. Englisch - verfügbar zu machen, und auch die Zahlen- und Datumsformate an diese Sprachen anpasst.
- **Testen:** Zwar wurde die App schon durchgetestet (siehe 5.6. Testen), aber noch nicht von allen Betriebssystemen mit allen oder zumindest den beliebtesten Browsern. Möglicherweise gibt es je nach Browser und Betriebssystem Designunterschiede. Denkbar wäre auch, dass bestimmter JavaScript-Code nicht oder anders ausgeführt wird, da manche Browser strengere Limitierungen haben. Um eine Einheitlichkeit im Design zu gewährleisten, habe ich zwar eine `normalize.css` eingefügt, dennoch hat zu Beginn das Aussehen der Emojis je nach Betriebssystem stark variiert, woraufhin ich als einheitliche Emoji-Schriftart die kostenlose *Open-Moji*-Schriftart verwendet und auf alle Emojis angewendet habe. Gegebenenfalls ist dies auch bei anderen Design-, Userfreundlichkeits- oder Funktionalitätsfragen der Fall, weswegen ein Testen auf allen Betriebssystemen mit den fünf häufigsten Browsern sowie mit den jeweiligen mobilen Apps sinnvoll wäre.
- **Reichweite:** Die Reichweite von *ClimateQuest* ist noch nicht sehr groß, da es nur wenige User gibt. Indem man Werbung produziert und verbreitet, beispielsweise Werbeclips auf YouTube schaltet, GoogleAds einrichtet, Kooperationen mit Schulen oder Unternehmen eingeht – diesen Punkt bin ich bereits angegangen, indem ich ein Exposé für meinen Schulleiter verfasst habe, welches ein mögliches Konzept für eine Kooperation mit *ClimateQuest* darlegt – oder Social-Media-Kanäle erstellt, welche beispielsweise Klimaschutztipps beinhalten, kann dies verbessert werden. *Plant-for-the-Planet* hat mir beim bereits erwähnten Gespräch angeboten, die App auch durch ihr Social-Media-Team bzw. ihre schon bekannten Kanäle zu promoten; auch Prof. Dr. Dr. Höppe hat die App bereits auf seinem LinkedIn Kanal geteilt, ich kann auch auf seine Initiative hin bei einer Vorlesung von meiner App berichten. (siehe 3.1. Kooperationen)
- **Push Nachrichten für AppStore Apps:** Momentan funktionieren die Push-Nachrichten für aus dem Apple AppStore geladene Apps nicht, was geändert werden sollte.

---

<sup>14</sup> LearnDjango, Django Testing Tutorial, <https://learndjango.com/tutorials/django-testing-tutorial>, zuletzt aufgerufen: 28.12.2025

## 7. Quellen und Unterstützungsleistungen

### 7.1. Verwendete Bilder und Grafiken

Alle verwendeten Bilder, Grafiken und Tabellen habe ich selbst erstellt.

### 7.2. Internetseiten

- Cloudflare, HTTP/2 vs. HTTP/1.1: Wie wirken sie sich auf die Web-Performance aus?, <https://www.cloudflare.com/de-de/learning/performance/http2-vs-http1.1/>, zuletzt aufgerufen: 25.12.2025
- Cloudflare, Was ist ein Content Delivery Network (CDN)? | Wie funktionieren CDNs?, <https://www.cloudflare.com/de-de/learning/cdn/what-is-a-cdn/>, zuletzt aufgerufen: 28.12.2025
- Django, Database access optimization | Django documentation | Django, <https://docs.djangoproject.com/en/6.0/topics/db/optimization/>, zuletzt aufgerufen: 29.12.2025
- Klimasparbüchle, Geschäftsstelle der Nachhaltigkeitsstrategie, Ministerium für Umwelt, Klima und Energiewirtschaft, Baden-Württemberg, [https://um.baden-wuerttemberg.de/fileadmin/redaktion/m-um/intern/Dateien/Dokumente/2\\_Presse\\_und\\_Service/Publikationen/Klima/Klima-Sparbuechle-barrierefrei.pdf](https://um.baden-wuerttemberg.de/fileadmin/redaktion/m-um/intern/Dateien/Dokumente/2_Presse_und_Service/Publikationen/Klima/Klima-Sparbuechle-barrierefrei.pdf), zuletzt aufgerufen: 08.12.2025
- LearnDjango, Django Testing Tutorial, <https://learndjango.com/tutorials/django-testing-tutorial>, zuletzt aufgerufen: 28.12.2025
- LMU München, Kontaktseite - Münchener Zentrum für Nachhaltigkeit (MZN) - LMU München, <https://www.lmu.de/mzn/de/mitglieder/kontaktseite/julia-pongatz-a9fd8b57.html>, zuletzt aufgerufen: 29.12.2025
- LMU München, Kontaktseite - Münchener Zentrum für Nachhaltigkeit (MZN) - LMU München, <https://www.lmu.de/mzn/de/mitglieder/kontaktseite/henrike-rau-9bf6b4e3.html>, zuletzt aufgerufen: 29.12.2025
- LMU München, Themengebiet – LMU München, <https://www.lmu.de/de/die-lmu/struktur/zentrale-universitaetsverwaltung/presse-und-kommunikation-puk/expertenservice/themengebiet/wetter.html>, zuletzt aufgerufen: 29.12.2025
- Play Console Hilfe, Anforderungen an App-Tests für neue private Entwicklerkonten - Play Console-Hilfe, <https://support.google.com/googleplay/android-developer/answer/14151465?hl=de>, zuletzt aufgerufen: 30.12.2025
- PWABuilder, Publish your PWA to the iOS App Store | PWA Builder Blog, <https://blog.pwabuilder.com/posts/publish-your-pwa-to-the-ios-app-store/>, zuletzt aufgerufen: 30.12.2025
- Python, PEP 8 – Style Guide Lines for Python Code, <https://peps.python.org/pep-0008/>, zuletzt aufgerufen: 27.12.2025
- Statistisches Bundesamt, CO<sub>2</sub>-Fußabdruck der privaten Haushalte: 540 Millionen Tonnen, <https://www.destatis.de/DE/Themen/Gesellschaft-Umwelt/Umwelt/UGR/energiefluesse-emissionen/emissionen-energiefluesse.html>, zuletzt aufgerufen: 22.11.2025

- this:matters, Was ist Cloudflare eigentlich und welche Vorteile bietet es?, <https://thismatters.agency/blog/was-ist-cloudflare/>, zuletzt aufgerufen: 22.12.2025
- TUM, Startseite – Lehrstuhl für Umwelt- und Klimapolitik, <https://www.hfp.tum.de/environmentalpolicy/startseite/>, zuletzt aufgerufen: 29.12.2025
- Umweltbundesamt, Treibhausgasemissionen stiegen 2021 um 4,5 Prozent, <https://www.umweltbundesamt.de/presse/pressemitteilungen/treibhausgasemissionen-stiegen-2021-um-45-prozent>, zuletzt aufgerufen: 22.11.2025
- Uni Bibliothek LMU, Content-Select: Clean Code - Refactoring, Patterns, Testen und Techniken für sauberen Code, <https://content-select-com.emedien.ub.uni-muenchen.de/de/portal/media/view/5c85864b-fe70-4252-afa0-6037b0dd2d03>, zuletzt aufgerufen: 28.12.2025